

Task 1 - Web App QA & Debug Report

Anshu Jain, 30 Sep 2026

App tested: RealWorld "Conduit" (the Medium clone). The demo.realworld.io link from the assignment is dead now (404, and its API gives a 530), so I looked around and found the project moved to demo.realworld.show (Angular frontend) with the backend at api.realworld.show/api. I used that live demo.

What I did: went through sign up, login, create/edit/delete article, comments, settings, logout, and using the same account in two browsers. I clicked through the UI in Edge (automated with Playwright so I could get screenshots) and also hit the API directly with small Python scripts to try the edge cases faster: blank and whitespace input, emoji, very long titles, duplicates, bad tokens, weird pagination values. I ran axe-core for accessibility and checked the response headers with curl. The scripts are in the task1/ folder if you want to re-run anything.

Short version: the normal happy path works fine, but the app basically trusts whatever the user types. You can end up with a post that can't be opened or deleted, sign up with a fake email and a 1 character password, or register the same email twice. Logging in on a second device quietly logs you out on the first one. I'd fix 1-3 first.

Bug table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Article with a blank (spaces only) or emoji-only title becomes a "ghost" post you can't open, edit or delete	1. Log in, click New Article 2. Title: a few spaces (or just 🚀🚀), fill the rest with anything 3. Publish 4. Go to your profile and click the post with no title	Expected: error "title can't be blank" for spaces; the emoji one should just work. Actual: it saves (201) with an empty slug "". The app goes to /article/ and shows a blank page (screenshot A). The post stays in the profile list forever. The emoji title got slug -1 and !!! got -2.	High	The required check doesn't trim, so spaces pass. The slug is made by stripping everything that isn't a letter/number, which leaves nothing. Details in the root cause section.
2	Sign up doesn't validate email format or password length	1. Sign up page 2. Email not-an-email, password 1 3. Click Sign up	Expected: "invalid email" / "password too short". Actual: account gets created and I'm logged in (screenshot B). The button is only disabled when fields are completely empty. The API also takes a username that is just spaces.	High	Only "is it present" checks, on both the Angular form and the backend. Without a real email there's no way to do password reset later.
3	Same email (and username) can be registered twice	1. POST /api/users with x@example.com / password "first" 2. Same again with password "second" 3. Log in with each password	Expected: 422 "email has already been taken". Actual: both return 201 and both passwords log in, but into two <i>different</i> accounts with different data. Which one you get depends on the password.	High	No unique check on email/username. The hosted demo keeps users sandboxed from each other, which might be why, but I'd still want to check this on a self-hosted backend before calling it demo-only.
4	Logging in on a second device logs you out on the first, with no message	1. Log in on browser 1 2. Log in to the same account on browser 2 3. On browser 1 click New Article (or publish a draft)	Expected: both stay logged in, or at least tell me I was signed out. Actual: browser 1's token stops working (401) and it just lands on the Sign in page (screenshot C). A draft in progress is lost. The API error says "token is missing" even though a token was sent.	Medium	Every login creates a new token and kills the old ones. The frontend treats any 401 as "logged out" and doesn't explain anything.

5	Security basics missing: no security headers, token in localStorage, CORS open to any site	1. curl -I https://demo.realworld.show 2. DevTools > Application > Local Storage 3. OPTIONS request to the API with Origin: https://evil.example	Expected: CSP, X-Frame-Options, HSTS etc, and CORS limited to the app's own domain. Actual: none of those headers. Token is stored in localStorage.jwtToken so any injected script can read it. API echoes back any origin with allow-credentials true. Profile image URL accepts javascript:alert(1).	Medium	Default nginx setup and a "reflect the origin" CORS config. The markdown rendering does sanitize XSS right now (I tested it), but if that ever breaks there is nothing else stopping it.
6	Accessibility problems on the home page	1. Open home page 2. Run axe DevTools (or Lighthouse)	Expected: no serious WCAG AA issues. Actual: 4 images without alt text, 4 links with no readable name (the avatar links), 40 color contrast failures (grey dates and "Read more..."), no lang on the html tag, no main landmark.	Medium	Template markup without alt/aria attributes and a very light grey that doesn't meet 4.5:1 contrast. A screen reader just reads "link, image" for each author.
7	Tags aren't cleaned up, "qa", "QA" and " qa " are 3 separate tags	1. New article with tags qa, QA, qa 2. Look at the article / popular tags	Expected: one tag "qa". Actual: [" qa ", "QA", "qa"], so filtering by tag misses posts.	Low	Tags saved exactly as typed, no trim/lowercase/dedupe.
8	Bad pagination values accepted silently	1. GET /api/articles?limit=-1&offset=-5	Expected: 422 or clamp to valid values. Actual: 200 with an empty list but articlesCount: 4, so the UI would say "no articles" when there are some. limit=abc gives "Invalid request body" which is confusing for a query param.	Low	Query params are type-checked but not range-checked.

Things I checked that were fine: empty forms get rejected, the XSS payloads I put in an article body (img onerror, javascript: links) were sanitized and nothing ran, titles over 100 chars are rejected, Japanese/Cyrillic/accented titles get proper slugs, writing without a token gives 401, and refreshing on a deep link like /article/some-slug works. Home page took about 3.5s to load on a cold browser, which is OK but worth watching (that's partly why I did the uptime monitor bonus).

One note on #4: my first attempt at this test actually made it look like it worked fine. It turned out I had logged in with not-an-email on the second browser, and because of bug #3 other people had registered that same email, so I was in a different account. Once I retried with a fresh account and logged the network calls, it reproduced every time.

Root cause analysis - #1 ghost articles

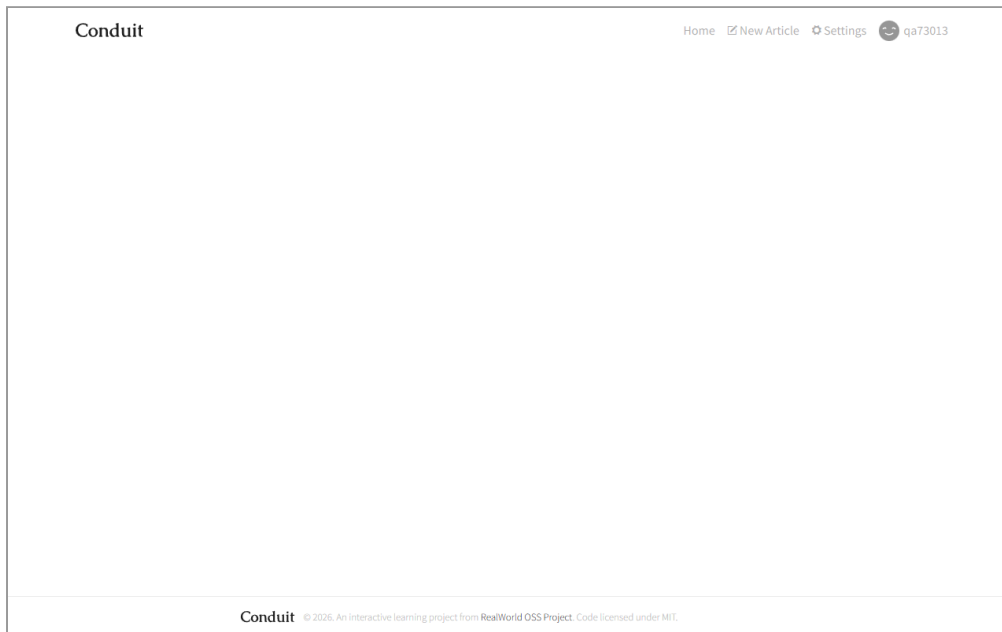
What happens: If the title is only spaces, or only emoji, the backend saves the article with an empty slug. The slug is what goes in the URL for everything (view, edit, delete are all `/articles/<slug>`). With an empty slug the URL turns into `/articles/`, which is the "list all articles" endpoint. So opening it shows a blank page, and the delete call gets a 307 redirect and never actually deletes anything. The post still shows up in the author's feed, so it's stuck there.

Why: It's two small things together. First, the "title is required" check looks at the raw string, and " " isn't empty, so it passes. Nothing trims it. Second, the slug function assumes the title will always have at least one letter or number in it. It strips everything else, so spaces or emoji give back an empty string, and nobody checks for that. The code that makes slugs unique then treats "" as a real slug, which is why the next bad titles got -1 and -2. Each piece works on normal input, it's the combination at the edge that breaks.

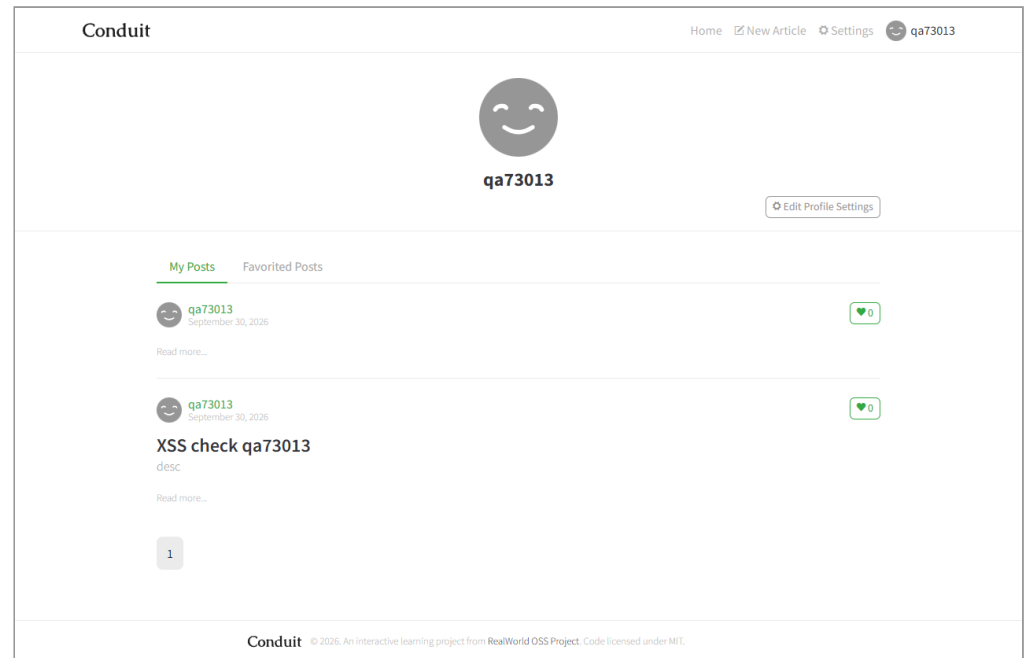
How I'd fix it:

1. Trim title/description/body on the server and return a 422 if they're empty after trimming. Add the same check to the Angular form so Publish stays disabled, but the server is what matters.
2. If the slug comes out empty, fall back to something generated like `article-7f3k9q`. Emoji titles are valid, they just need a slug.
3. Add a DB constraint so slug can't be empty (on top of the existing unique index), so this can't come back through some other code path.
4. One-off script to fix existing rows where slug is "" or looks like -1, -2.
5. Add regression tests for titles like spaces, 🚀, !!!, 日本語 and duplicate titles. Each one should either be rejected or get a slug you can open and delete.

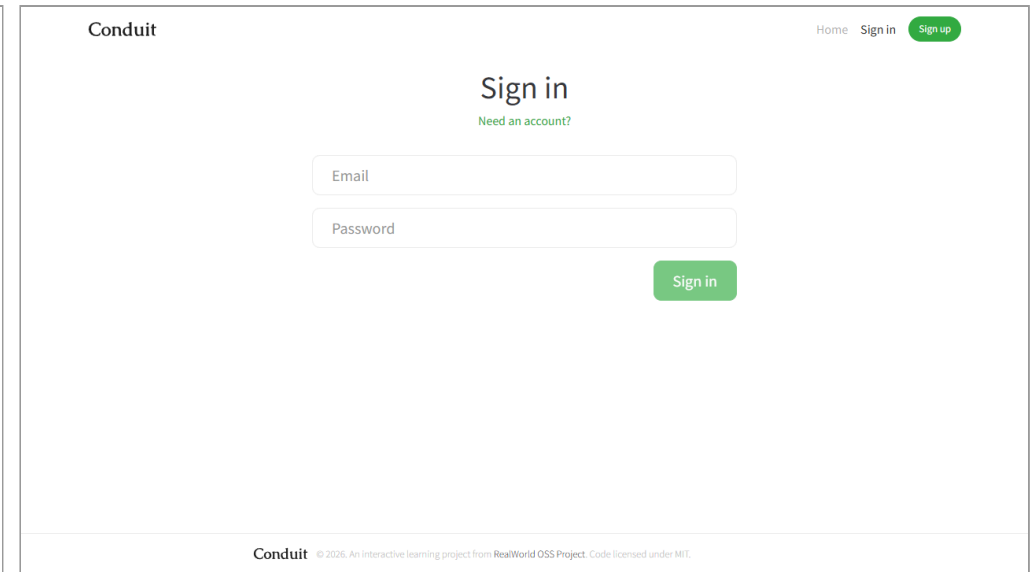
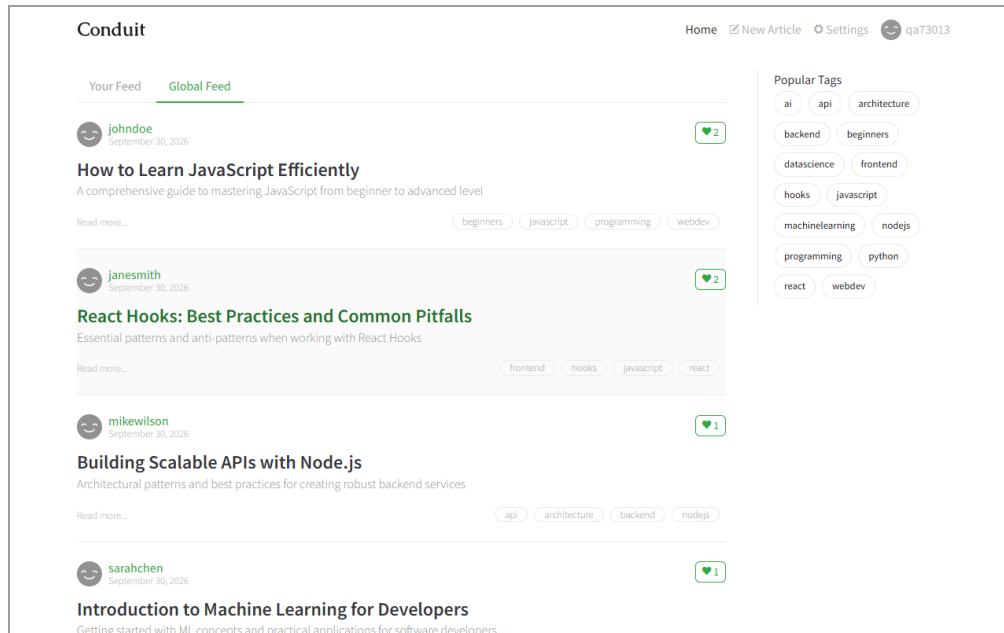
Screenshots



A - after publishing a title of just spaces, the app goes to `/article/` and shows nothing



A2 - the ghost post (no title) sitting in my profile above a normal one



C - browser 1 after the same account logged in on browser 2, back on Sign in with no explanation

B - signed up and logged in with email "not-an-email" and password "1"

Assumptions: this is the public shared demo and users are sandboxed from each other, so I only used throwaway accounts and didn't do any load testing. Severities are how I'd rate them if this were our own production app.